

# AutoOps AI Viva Guide

What the system does, how a request moves through it, how the risk score is calculated, and the questions a panel is likely to ask, with short answers.

## The system in one sentence

An IT support chat that can fix a user's computer problem, but only when it is safe to act, and it proves the fix worked by reading the machine again.

## The novelty: two claims

The AI may **propose** a fix. It never decides whether the fix may run, and it never decides whether the fix worked.

### Claim 1

#### Weak evidence costs autonomy

If the knowledge base has no good article for the problem, the risk score goes up. The same fix can run automatically with strong evidence and need a human with weak evidence.

### Claim 2

#### Completion is not resolution

A command finishing without error proves only that it ran. After every action the system reads the machine's real state and compares it with the state before.

## Workflow

Dashed box: the AI works here. **NOVEL** marks the research contribution.

1

#### User reports a problem in the chat

"My computer is slow." They can report from any device, including a phone.

↓

2

#### AI understands and searches the knowledge base **AI**

The classifier (the AI, or a rule-based fallback) gives a confidence. Retrieval gives a similarity score for the best matching approved article.

↓

3

**AI proposes one action from the catalogue** AI

Only an action ID from an allow-list of 25 actions, never a free command.

↓

4

**Risk engine scores it** NOVEL

Five-factor formula in plain Python, then safety rules that can only raise the level. The AI never writes the score.

↓

**LOW ·  $\leq 1.6$** 

Runs without an approval token when the user clicks Run

**MEDIUM ·  $\leq 2.2$** 

The user or permitted support staff must approve

**HIGH ·  $> 2.2$** 

Blocked unless a qualified second person approves

↓

5

**Approval token**

Single use, valid 15 minutes, tied to the exact action, parameters and target machine. If any of them changes, the approval stops working and the action is scored again.

↓

6

**Pre-check: is the problem real on this machine?** NOVEL

A disk cleanup is refused when the disk has plenty of space. A network reset is refused when the network works.

↓

7

**Agent on the user's PC runs the action**

The agent asks the server for work. It receives an action ID and looks it up in its own catalogue. If the machine never answers, a ticket is raised for a technician.

↓

8

**Post-check: read the machine again** NOVEL

Compare the state before and after. Example: after a cleanup, free disk space must have gone up.

↓

**VERIFIED FIXED**

Reported as resolved

**VERIFIED NOT FIXED**

Undo the change, verify the undo, else escalate

**INCONCLUSIVE**

Not reported as fixed. Sent to a person

↓

9

**AI explains the result to the user** AI

Only in words, from what the checks found. It may not claim a reading it was not given.

↓

**Audit log**

Every step is recorded: who, what, which risk version, and the result.

## How the risk score is calculated

$$S = 0.30 \times I + 0.20 \times C + 0.20 \times E + 0.15 \times R + 0.15 \times A \quad (\text{each factor 1 to 3})$$

| FACTOR                     | WEIGHT | WHERE THE MARK COMES FROM                                                                                                          |
|----------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------|
| I · Impact                 | 0.30   | Catalogue risk of the action. A read-only diagnostic is always 1.                                                                  |
| C · Diagnostic uncertainty | 0.20   | Confidence from the AI or the fallback classifier: $\geq 0.85 \rightarrow 1$ , $\geq 0.65 \rightarrow 2$ , lower $\rightarrow 3$ . |
| E · Evidence weakness      | 0.20   | Best knowledge-base match: $\geq 0.80 \rightarrow 1$ , $\geq 0.70 \rightarrow 2$ , lower or none $\rightarrow 3$ .                 |
| R · Irreversibility        | 0.15   | Read-only or has a rollback $\rightarrow 1$ . No rollback $\rightarrow 2$ , or 3 if high risk.                                     |
| A · Affected scope         | 0.15   | Only the user's own machine $\rightarrow 1$ . Network or services $\rightarrow 2$ .                                                |

Confidence and evidence are stored as "3 = best" and turned around only inside the formula:  $C = 4 - \text{confidence rating}$ .

### Claim 1 in numbers

The same action (impact 2, no rollback, confident diagnosis, own machine). Only the evidence changes.

| EVIDENCE     | I    | C    | E    | R    | A    | SCORE | RESULT                        |
|--------------|------|------|------|------|------|-------|-------------------------------|
| Strong match | 0.60 | 0.20 | 0.20 | 0.30 | 0.15 | 1.45  | <b>Low</b> runs automatically |
| No match     | 0.60 | 0.20 | 0.60 | 0.30 | 0.15 | 1.85  | <b>Medium</b> needs a human   |

### Safety rules (overrides)

They are checked after the score. They can only raise the level, never lower it.

- Missing required evidence (read-only diagnostics are exempt)
- Privileged security action
- Irreversible change to a shared resource
- No rollback on a critical resource

- The action's own catalogue risk is the minimum level

## Evaluation results

32 labelled scenarios × 3 conditions × 3 repeats = 288 runs, on the simulated driver so faults can be injected the same way every time. No AI model is called, so every run gives the same result.

| MEASURE                             | A • ADVICE ONLY | B • SAME APPROVAL FOR ALL | C • RISK-ADAPTIVE (MINE) |
|-------------------------------------|-----------------|---------------------------|--------------------------|
| Unsafe actions executed (of 36)     | 0               | 36                        | 0                        |
| Human approvals needed (of 96)      | 0               | 96                        | 72                       |
| Actions executed                    | 0               | 93                        | 57                       |
| Verified resolved                   | 0               | 69                        | 36                       |
| Injected faults reported as success | 0               | 0                         | 0                        |
| Rollbacks attempted / verified      | 0 / 0           | 6 / 3                     | 6 / 3                    |
| Audit trail complete                | 100%            | 100%                      | 100%                     |

Main result: compared with B, C executed 0 of 36 unsafe actions instead of 36, and needed 24 fewer human approvals.

## Questions the panel may ask

Short answers in plain words. Questions with a red edge are the hard ones. Know them best.

### The idea

#### 01 What problem does your project solve?

AI helpdesks either only give advice, or they act without checking whether it is safe and whether it worked. My system lets the AI suggest a fix, but measured evidence decides whether it runs. The system then checks the machine to confirm the result.

### 02 What exactly is new in your work?

Two things. First, the amount of freedom the system gets depends on the strength of the evidence, so weak evidence means more human approval. Second, success is decided by reading the real machine state, not by the command's exit code.

### 03 Why not let the AI decide whether an action is safe?

An AI can be confidently wrong, and it cannot measure its own reliability. Letting it score its own suggestion is like a student marking their own exam. The score is calculated by plain Python from five inputs: a retrieval similarity, a classifier confidence, and three fixed properties of the action.

### 04 But the confidence comes from the AI. Isn't the AI scoring itself after all?

Partly, and my thesis lists it as a limitation (Table 4.1). But confidence is only one of five factors, with weight 0.20. The AI cannot change the formula, the thresholds or the safety rules. And a high confidence cannot lower the risk below the action's catalogue level or below what a safety rule requires.

### 05 How is this different from normal automation with approval?

Normal automation uses the same approval rule for every action. That is condition B in my evaluation. It either asks a human every time or not at all. Mine changes the route for each request based on risk, and in the evaluation it blocked all 36 unsafe cases while needing 24 fewer approvals.

### 06 What is the role of the AI in your system?

Three jobs: understand the problem (and say how confident it is), propose an action from a fixed list, and explain the result in simple words. It does not calculate the risk score, approve anything, run commands, or decide whether the fix worked.

## Risk scoring

### 07 Explain your risk formula.

Five factors from 1 to 3: impact, diagnostic uncertainty, evidence weakness, irreversibility and scope. They are weighted 0.30, 0.20, 0.20, 0.15 and 0.15 and added up. Up to 1.6 is low and runs automatically, up to 2.2 is medium and needs the user's approval, and above that is high and goes to an expert.

### 08 Why these weights?

I chose them myself, and I say so in the thesis. The literature supports the five factors (Chapter 3), but it does not give the weights. Impact is highest because possible harm matters most. The weights are transparent and configurable, and every assessment stores the weights and policy version, so any decision can be replayed. Calibrating them with experts is future work.

#### 09 Why "4 minus confidence"?

Confidence is stored the natural way: 3 means most confident. The formula needs "high means risky", so the value is turned around only inside the calculation.

#### 10 Your screen shows "High risk" with a score of 1.55. Isn't that a bug?

No. By score alone it would be low, but a safety rule applied: the action changes the system and no evidence supported it. Safety rules can only raise the level, so it was blocked and sent to an expert. That is claim 1 at its strongest.

#### 11 Why do diagnostics run without evidence?

A diagnostic only reads the machine. Reading the machine is how evidence is collected. If diagnostics needed evidence first, the system could never collect any.

#### 12 How is evidence quality measured?

By the similarity score of the best matching approved knowledge-base article: 0.80 or more is strong, 0.70 or more is partial, and lower is weak. I use only the best match, because three weak matches are the same weak claim repeated.

#### 13 What if the risk engine fails?

It fails closed. The action is marked not executable. It never runs by default.

#### 14 Can the system learn from solved problems?

A solved problem becomes a knowledge draft. Only after a human reviewer approves it does it become searchable. Otherwise the system would base its evidence on its own unchecked guesses.

## Verification

#### 15 How do you verify that a fix worked?

Every action has a contract. The pre-check asks whether the problem is real. The post-check reads the state after the action and tests a condition. For example, a disk cleanup must increase free space.

#### 16 Why is the exit code not enough?

Scenario EV-31: disabling Teams at startup succeeds, but Teams registers itself again straight away. The command succeeded and achieved nothing. Only reading the real state shows that.

### 17 What are the possible results?

Verified fixed, verified not fixed, or inconclusive. Inconclusive is never reported as fixed, and it is not reported as failed either, because nobody could observe it. It goes to a person.

### 18 What happens when a fix fails?

If the action has a rollback, the system runs it and then verifies the rollback like any other action. A restore counts only when it is verified. If there is no rollback, or the rollback cannot be verified, the problem goes to a human.

### 19 A diagnostic only reads. How can it fail a check?

The system checks that a read-only action changed nothing. Some values change by themselves on a live machine, like the process list and the DNS cache. Those are excluded. Disk space gets a measured allowance of 100 MB, which is ten times the normal drift and far below what a cleanup frees.

### 20 Give an example of a pre-check refusing an action.

A disk cleanup runs only if free space is below 20 GB or below 10% of the disk. A network reset runs only when the machine has no connection. Otherwise a report of "slow internet" could cause a network reset and a restart on a machine that was fine.

## Security and architecture

### 21 Could the AI run a dangerous command?

No. A driver accepts only an action ID from an allow-list, never a command string. The agent on the PC also checks the ID against its own catalogue and refuses anything unknown.

### 22 How does the agent talk to the server?

The agent asks for work, and the server never connects to the PC. Each machine is registered by an administrator and gets a device ID and a secret. The secret is shown once and stored only as a hash.

### 23 Can someone approve one action and run another?

No. The approval token is single-use, valid for 15 minutes, and its fingerprint covers the action, the parameters and the target machine. If any of them changes, the token stops working and the action is scored again.

### 24 How do you know who the user is?

Every request needs a login token (JWT). The user is taken from the token, never from the request body. Otherwise anyone could act, and approve, in someone else's name.

### 25 What if the user's machine is off?

No action is offered, and a high-priority ticket is created and assigned to a technician, naming the machine. The result stays "unknown", not "failed", because nothing was observed.

### 26 Did risk and verification change when you added real Windows machines?

No. Only a new driver was added. Risk, approval, checks, rollback and audit stayed the same, which shows the design is modular.

## Evaluation

### 27 How did you evaluate the system?

32 labelled scenarios under 3 conditions: A (advice only), B (same approval for every action) and C (my risk-adaptive system). Each ran 3 times, 288 runs in total. Faults were injected to test whether failures are caught.

### 28 What are the main results?

Unsafe actions executed: B 36, C 0. Human approvals: B 96, C 72, so 24 were avoided. Injected faults reported as success: 0 in every condition. Audit trail complete in 100% of runs.

### 29 B resolved more problems than C (69 vs 36). Isn't B better?

No. Compare scenario by scenario. The 19 scenarios that both B and C executed ended with the same verdicts and the same final states. The difference is the 12 unsafe high-risk scenarios: B ran them, C blocked them. B's higher number comes from running actions that should not have run.

### 30 Risk classification accuracy is 100%. Isn't that suspicious?

Yes, and I say so. I labelled the scenarios myself, so 100% shows the code matches its own specification. It is a consistency check, not agreement with experts. Independent expert labelling is required by thesis 6.3 and is the next step.

### 31 Was the evaluation run on real computers?

The evaluation used the simulated driver, because injecting the same fault every time needs control of the machine. Separately, I showed the full path on one real Windows machine through the agent. I keep these two claims separate, and one Windows machine is a limit on how far the result generalises.

## Limitations

### 32 What if the agent lies about the machine's state?

The device reports its own state, and verification trusts it. A hacked agent could claim a fix that did not happen. Solving that needs attested measurement, which is outside this project. What I do have is an audit record tied to one device credential that can be revoked.

### 33 Are there actions that can never succeed?

Yes, two, and both fail safe. `reset_winsock` never runs on Windows because the network stack's health cannot be read, so its pre-check refuses. `restart_explorer` always ends "inconclusive" because a restarted shell looks the same as before, so it escalates.

### 34 Your knowledge base has only 30 articles. Is that enough?

It is small, and all articles are about Windows. That is why evidence is often weak and more actions need approval. That is claim 1 working as designed. More articles would give the system more autonomy.

### 35 Why no vector database or LangChain?

With 30 articles, they were not needed. Retrieval calls the Gemini SDK directly with `gemini-embedding-001` and searches in memory. It searches the first problem the user reported together with their newest message, so one side comment does not change the evidence.

### 36 Condition B is "approve everything". Did real people approve?

No. B simulates a person who approves every request without checking. That is how it tests the worst case of uniform approval. It is not a measurement of real human behaviour, and the thesis says so.

### 37 Did you test it with real users?

No recruited participants, so I make no claims about usability or trust. A participant study is future work, to check whether people understand the risk explanations and do not approve everything without reading.

### 38 Can someone change the audit log?

It records every step, but it is not cryptographically tamper-proof. Someone with database access could edit it. Making it tamper-evident, for example with hash chaining, is future work.

### 39 Which states cannot be read on Windows?

Pending Windows updates, which need the Windows Update COM API. So an update action ends "inconclusive" on a real machine and is not reported as fixed.

#### 40 What is future work?

Independent expert labels, calibrating weights and thresholds, a larger knowledge base, tests on more Windows builds and operating systems, a user study, attested device measurement, and a tamper-evident audit log.

## If you remember one answer

---

"Other systems let the AI decide and trust the exit code. In mine, the AI only proposes. Measured evidence decides how much freedom it gets, and the system proves the result by reading the machine's real state."